

# ILLYRIAN VAULT

## Interview Q&A Guide

Prepared by: Orest Zogju

June 2026

---

32 questions covering project overview, architecture, security, database, C#/.NET specifics, design decisions, testing, and behavioral topics.

# Table of Contents

- Section 1: Project Overview (Q1-Q4)
- Section 2: Architecture & Design Patterns (Q5-Q8)
- Section 3: Security (Q9-Q15)
- Section 4: Database & Data (Q16-Q18)
- Section 5: C# / .NET Specifics (Q19-Q22)
- Section 6: Design Decisions & Trade-offs (Q23-Q26)
- Section 7: Testing & Quality (Q27-Q28)
- Section 8: Soft Skills / Behavioral (Q29-Q32)

# Section 1: Project Overview

## Q1: Can you give a high-level overview of your project?

Illyrian Vault is a local, offline password manager I built for Windows Desktop using C# .NET 10 and WPF. It stores all credentials in a SQLCipher-encrypted SQLite database on the user's machine — nothing is ever sent to the cloud. The architecture follows strict MVVM using CommunityToolkit.Mvvm source generators, with separate Auth and Main windows. It supports four entry types (Login, Card, Identity, Secure Note), three visual themes, two languages (English and Albanian), and includes features like a built-in password generator, breach checking via Have I Been Pwned, auto-locking on idle, and an auto-updater via GitHub Releases.

---

## Q2: Why did you choose WPF over other UI frameworks?

WPF was the right choice for several reasons:

- **(1)** The app is Windows-only — it doesn't need cross-platform support.
- **(2)** WPF has the strongest MVVM ecosystem with mature data binding, commands, and DataTemplate-based composition.
- **(3)** It supports borderless windows with AllowsTransparency for the custom UI design I was targeting from an HTML/CSS mockup.
- **(4)** CommunityToolkit.Mvvm eliminates most boilerplate through source generators, making the developer experience comparable to modern frameworks.

I considered alternatives like MAUI but its Windows desktop support was less mature, and Avalonia would add complexity for a single-platform app.

---

## Q3: What was the most challenging part of building this project?

The security layer was the hardest. Getting key derivation, memory management, and encryption right requires understanding low-level .NET runtime behavior. For example, I had to use GC-pinned byte arrays to prevent the garbage collector from moving (and duplicating) sensitive key material during compaction. I also had to extract passwords from WPF's SecureString through a BSTR → pinned char[] → pinned UTF-8 byte[] pipeline without ever creating a managed System.String.

Another challenge was the recovery key flow — it required wrapping the master key with a recovery-derived key, then on recovery, unwrapping it, re-keying the entire SQLCipher database, AND re-encrypting every individual AES-GCM field with the new key.

---

## Q4: How does your project handle the trade-off between security and usability?

Several carefully balanced mechanisms:

- The clipboard auto-wipes after 15 seconds — long enough to paste but short enough to limit exposure.
- The idle lock has configurable timeouts (1-30 minutes) so users choose their own risk tolerance.
- The brute-force defense uses progressive backoff (not instant lockout) so a typo doesn't lock you out, but 10+ failures trigger a 5-minute lockout.
- Password strength is enforced with a minimum score of 3/5 during registration but shown visually so users understand why their password was rejected.

- The recovery key flow requires downloading the key file before proceeding — forcing users to save it while not making it so cumbersome they skip it.

## Section 2: Architecture & Design Patterns

### Q5: Explain the MVVM pattern and how you implemented it.

MVVM separates the application into three layers. The **Model** layer (PasswordEntry, VaultUser, payload types) holds pure data. The **ViewModel** layer (MainViewModel, LoginViewModel, etc.) contains all business logic, state, and commands — it exposes properties the View binds to but has zero knowledge of any UI element. The **View** layer (XAML + code-behind) contains only UI layout and binds to ViewModel properties via WPF's data binding engine.

I used CommunityToolkit.Mvvm's **[ObservableProperty]** attribute which source-generates INotifyPropertyChanged boilerplate, and **[RelayCommand]** which generates ICommand implementations. Navigation uses ViewModel-first composition: AuthViewModel swaps its CurrentViewModel property between Login/Register/Recovery ViewModels, and DataTemplates in App.xaml automatically resolve the correct View for each ViewModel type.

---

### Q6: How do you handle dependency injection in this project?

I used manual service location through static properties on the App class rather than a DI container. App.Encryption, App.Database, App.Theme, and App.Localization are singleton services initialized once at startup. The reason is simplicity — this is a desktop app with a small, fixed set of services, and introducing a DI container (like Microsoft.Extensions.DependencyInjection) would add complexity without proportional benefit.

ViewModels receive their dependencies through **constructor injection**: LoginViewModel takes EncryptionService and DatabaseService as constructor parameters, MainViewModel additionally receives the session key and username. This is still testable and explicit about its dependencies.

---

### Q7: How does the polymorphic entry system work?

Each vault entry has a **Category** (Login/Note/Card/Identity) and an **EncryptedPayload** field. The **IEntryPayload** marker interface is implemented by four concrete payload classes (LoginPayload, CardPayload, IdentityPayload, NotePayload), each with category-specific fields.

When an entry is saved, the payload object is JSON-serialized and AES-GCM encrypted into EncryptedPayload. When displayed, it's decrypted, deserialized back to the concrete type, and rendered through a **DataTemplateSelector** (PayloadViewSelector for read-only, PayloadEditSelector for edit mode). Login entries also maintain flat fields (Username, EncryptedPassword, URL) for backward compatibility and search.

---

### Q8: Explain the observer pattern in your codebase.

I use C# events extensively as an observer pattern implementation. For example:

- **AuthViewModel.LoginSucceeded** (Action<string>) fires when authentication completes — the AuthWindow subscribes and opens MainWindow.
- **EntryViewModel.SaveCompleted** fires after a successful save, and MainViewModel subscribes to show a toast notification.
- **PasswordGeneratorViewModel.PasswordAccepted** fires when the user clicks "Use This Password", and both AddEntryViewModel and EntryViewModel subscribe to inject the generated password.

This keeps ViewModels decoupled — they don't reference each other directly, only communicate through events.

## Section 3: Security

### Q9: Walk me through your encryption strategy.

There are **two layers** of encryption. First, **SQLCipher** provides full-database AES-256 encryption — every page of the SQLite file is encrypted at rest. The key is derived from the user's master password via PBKDF2 with 600,000 iterations (SHA-512 for new vaults, SHA-256 for legacy). I pass a raw 32-byte hex key to SQLCipher via PRAGMA key, bypassing its internal KDF since I handle derivation myself.

Second, individual sensitive fields (passwords, structured payloads) are encrypted with **AES-256-GCM** using a fresh 96-bit nonce per operation. The wire format is [12B nonce][16B GCM tag][N B ciphertext] encoded as Base64. GCM provides **authenticated encryption** — tampering with the ciphertext causes decryption to fail, preventing silent data corruption.

---

### Q10: Why 600,000 PBKDF2 iterations? How did you decide that number?

600,000 is the **OWASP 2023 recommended minimum** for PBKDF2-SHA256. At this iteration count, deriving a key takes approximately 400ms on a modern CPU — noticeable but acceptable for a login operation that happens once per session. For an attacker with a high-end GPU, this translates to roughly 2-4 million guesses per second, making a 60-bit entropy passphrase computationally infeasible within a human lifetime.

For v2 vaults I use **SHA-512 instead of SHA-256** because GPU mining hardware is optimized for SHA-256 (32-bit operations), while SHA-512 uses 64-bit words that current 32-bit shader units process less efficiently, raising the attacker's cost by approximately 1.5-2x.

---

### Q11: What is GC pinning and why is it critical in your security model?

The .NET garbage collector periodically compacts the managed heap by physically moving objects, which leaves stale copies of data at previous memory addresses. For sensitive key material, this means the GC could create multiple copies of your encryption key scattered across RAM.

**GC pinning** (GCHandle.Alloc with GCHandleType.Pinned) tells the GC "do not move this object." My **PinnedBuffer** struct wraps a pinned byte[] and implements IDisposable — on disposal, it uses **CryptographicOperations.ZeroMemory** (a volatile write the JIT cannot optimize away) to overwrite all bytes with zeros, then releases the pin. This ensures the key exists at exactly ONE memory address for its entire lifetime and is deterministically zeroed.

---

### Q12: How does SecureString extraction work and why is it important?

WPF's PasswordBox stores the user's password in a SecureString, which is DPAPI-encrypted in kernel memory. The standard approach (.Password property) decrypts it into a managed System.String, which lives on the GC heap and **cannot be zeroed** (strings are immutable in .NET). My approach avoids this entirely:

- **Step 1:** Marshal.SecureStringToBSTR decrypts into a native-heap BSTR (UTF-16).
- **Step 2:** Marshal.Copy transfers it to a GC-pinned char[] — no String created.
- **Step 3:** Encoding.UTF8.GetBytes converts to a GC-pinned byte[].
- **Step 4:** The char[] is zeroed via CryptographicOperations.ZeroMemory and its pin released.
- **Step 5:** The BSTR is zeroed and freed via Marshal.ZeroFreeBSTR.

The resulting PinnedBuffer contains UTF-8 password bytes that have **never existed as a managed String**.

---

### Q13: How does the recovery key system work?

During registration, I generate a 16-byte random recovery key formatted as **ILVT-XXXX-XXXX-XXXX-XXXX-XXXX-XXXX-XXXX**. This key is used to derive a wrapping key via PBKDF2-SHA512 (600k iterations with its own salt). The current master key is then AES-GCM encrypted ("wrapped") with this wrapping key. The wrapped master key and its salt are stored in vault.meta. The recovery key itself is **NEVER stored** — the user downloads it as a text file.

**Recovery flow:** User enters recovery key → wrapping key re-derived → wrapped master key decrypted → vault opened with old master key → new master key derived from new password → SQLCipher database re-keyed (PRAGMA rekey) → ALL individual AES-GCM fields re-encrypted with new key → vault.meta updated with new salt, hash, and wrapped key.

---

### Q14: How do you prevent timing attacks?

I use **CryptographicOperations.FixedTimeEquals** for all security-critical comparisons. Standard equality comparison (== or .Equals) short-circuits on the first differing byte — an attacker measuring response times across many attempts could determine the correct hash byte-by-byte. FixedTimeEquals always compares ALL bytes regardless of where differences occur, making the execution time constant.

I use this in the verification hash comparison during login and in the ClipboardGuard's ownership hash comparison (the latter is defense-in-depth since clipboard content isn't secret).

---

### Q15: How does the breach check work without exposing the password?

I use the **Have I Been Pwned k-anonymity API**. The flow is:

- **(1)** Hash the password with SHA-1.
- **(2)** Send ONLY the first 5 hex characters of the hash to the API.
- **(3)** The API returns all known breach hashes that start with those 5 characters (~500 results).
- **(4)** Compare the remaining 35 characters of the hash locally against the returned list.

The full password and full hash **never leave the device**. The API cannot determine which entry I'm actually checking because hundreds of possible passwords share the same 5-character prefix. I also check network availability first and handle errors gracefully — a failed breach check never blocks the user.

## Section 4: Database & Data

### Q16: Explain your database architecture.

Each user gets an isolated SQLite database at `%LOCALAPPDATA%\IllyriaVault\Profiles\{username}\vault.db`, encrypted with SQLCipher AES-256. The schema has three tables:

- **Users** — single row per vault (username, salt, verification hash, recovery hash).
- **PasswordEntries** — all vault entries (title, username, encrypted password, URL, notes, category, encrypted payload, favorite flag, timestamps).
- **PasswordHistory** — auto-archived previous passwords linked via foreign key to PasswordEntries with ON DELETE CASCADE.

I use **WAL journal mode** for better concurrent read performance and set `synchronous=NORMAL` for a balance between safety and performance.

---

### Q17: How do you handle schema migrations?

I use a pragmatic try/catch approach for ALTER TABLE migrations. After CREATE TABLE IF NOT EXISTS for all tables, I run ALTER TABLE statements wrapped in try/catch blocks — if the column already exists, SQLite throws an exception which I catch and ignore. For example:

```
try { ALTER TABLE Users ADD COLUMN Username TEXT NOT NULL DEFAULT ''; }  
catch (SQLException) { /* column already exists */ }
```

For larger-scale apps I'd recommend a proper migration framework like EF Core Migrations, but for a local single-user database this approach works well and is easy to reason about.

---

### Q18: Why SQLCipher instead of application-level encryption only?

**Defense in depth.** Application-level AES-GCM encrypts individual fields, but SQLite's metadata (table names, column names, row counts, index structures) would be visible in a plaintext database file. SQLCipher encrypts the ENTIRE database file — every page, including schema information. An attacker who copies the .db file sees nothing useful without the key.

The two layers complement each other: SQLCipher protects the database structure and provides at-rest encryption, while field-level AES-GCM provides an additional layer for the most sensitive values (passwords, card numbers) with per-field nonces and authentication tags.

## Section 5: C# / .NET Specifics

### Q19: How do source generators work in CommunityToolkit.Mvvm?

CommunityToolkit.Mvvm uses **C# source generators** that run at compile time. When I declare **[ObservableProperty] private string \_errorMessage**, the generator creates a public ErrorMessage property with get/set that calls SetProperty (which triggers INotifyPropertyChanged). [NotifyPropertyChangedFor(nameof(HasError))] adds a PropertyChanged notification for HasError when ErrorMessage changes.

**[RelayCommand]** on a method like **private async Task UnlockAsync()** generates an AsyncRelayCommand property named UnlockCommand with CanExecute support. The class must be marked **partial** because the generator creates a companion partial class with the generated code. This eliminates hundreds of lines of manual property-change boilerplate.

---

### Q20: Explain how async/await works in your codebase.

Most database and I/O operations are async to keep the UI thread responsive. For example, LoginViewModel.UnlockAsync() performs PBKDF2 derivation (CPU-intensive, ~400ms) and database operations without freezing the login button. WPF's RelayCommand handles async commands — while the task runs, CanExecute returns false (via IsBusy binding) so the button disables itself.

The brute-force backoff delay uses Task.Delay with a duration based on failure count. All database methods (OpenAsync, GetAllEntriesAsync, InsertEntryAsync) use async ADO.NET calls. The pattern is: set IsBusy=true → try/catch around async work → finally set IsBusy=false.

---

### Q21: What is the IAsyncDisposable pattern and how do you use it?

**IAsyncDisposable** is for cleaning up resources that require async operations to release. DatabaseService implements it because closing a SQLCipher connection should be awaited (CloseAsync, DisposeAsync) to ensure all pending WAL writes are flushed. The DisposeAsync method nulls the connection reference after disposal.

App.OnExit() calls **await Database.DisposeAsync()** to ensure clean shutdown. This is different from IDisposable which is synchronous — SecureMemory.PinnedBuffer uses IDisposable because zeroing memory and releasing a GC pin are synchronous CPU operations.

---

### Q22: How do you handle the PasswordBox MVVM binding limitation?

WPF's PasswordBox deliberately doesn't expose a bindable Password dependency property for security reasons (it would leave the password in the binding engine's cache). I created **PasswordBoxHelper** with two modes:

- **Secure mode:** An attached SecurePassword DependencyProperty that binds to a SecureString ViewModel property — the password never becomes a managed string.
- **Legacy string mode:** A BoundPassword attached property for RegisterViewModel's strength-score UX which needs string access for character analysis.

Each mode uses a **per-instance DependencyProperty guard** (not a static bool) to prevent re-entry loops when updating bi-directionally. This prevents subtle bugs when multiple PasswordBox instances exist simultaneously.

## Section 6: Design Decisions & Trade-offs

### Q23: Why manual service location instead of a DI container?

For a desktop app with 4 singleton services and no unit testing (yet), a DI container adds configuration overhead without proportional benefit. ViewModels receive dependencies through constructor parameters, which is still testable and explicit. If the project grew to need per-request scoping, interface-based abstraction, or significant unit testing, I'd introduce `Microsoft.Extensions.DependencyInjection`.

---

### Q24: Why store the session key in a static property on App?

The session key needs to be accessible from any ViewModel during the session (for encrypting/decrypting entries) but must be deterministically zeroed on lock/logout. A static property with pinned `byte[]` and explicit zero-on-clear is the simplest approach. The key is set once on login (`App.SetSessionKey` pins it), passed by reference to ViewModels, and cleared (`App.ClearSessionKey` zeros and unpins it) on lock/logout/exit. A DI-scoped lifetime would work too but adds indirection without improving security.

---

### Q25: Why pre-parse theme/localization ResourceDictionaries?

Parsing XAML at runtime takes **40-80ms** per `ResourceDictionary` on slower machines. `ThemeService` and `LocalizationService` parse all dictionaries once at startup and cache the `ResourceDictionary` objects. Switching themes/languages then becomes a cheap reference swap (Remove old → Add new) instead of re-parsing XML. The trade-off is slightly higher startup memory, but the dictionaries are small (a few KB each).

---

### Q26: Why not use Entity Framework Core?

EF Core adds significant dependency weight and doesn't integrate naturally with SQLCipher's PRAGMA-based key management. The app has only 3 tables with straightforward CRUD — hand-written SQL with parameterized queries is simpler, more transparent, and gives me full control over the SQLCipher configuration sequence (key → `cipher_page_size` → `cipher_hmac_algorithm` → etc., all of which must execute before any schema operations).

## Section 7: Testing & Quality

### Q27: How would you test the encryption service?

I'd write unit tests verifying:

- (1) DeriveKey with known inputs produces deterministic output.
  - (2) Encrypt followed by Decrypt returns the original plaintext.
  - (3) Decrypt with wrong key throws AuthenticationTagMismatchException (GCM authentication failure).
  - (4) Each Encrypt call produces different ciphertext (due to random nonce).
  - (5) CreateVerificationHash is deterministic for the same key.
  - (6) VerifyPassword returns true for correct password and false for incorrect.
  - (7) ScorePassword returns expected scores for known inputs (e.g., "abc" = 0, "Tr0ub4dor&3" = 5).
- 

### Q28: How do you handle error logging and crash resilience?

App.xaml.cs registers **three exception handlers**:

- **DispatcherUnhandledException** (UI thread) — sets Handled=true so the app continues.
- **AppDomain.UnhandledException** (non-UI threads) — logs but cannot prevent termination.
- **TaskScheduler.UnobservedTaskException** (unobserved async) — calls SetObserved to prevent crash.

All write to **%LOCALAPPDATA%\IllyriaVault\Logs\error-{date}.log** with rotation at 512KB. The user sees a generic "unexpected error occurred" message — never raw exception details or stack traces. This prevents information leakage while still providing diagnostic data for debugging.

## Section 8: Soft Skills / Behavioral

### Q29: What would you do differently if you started this project over?

I'd add **unit tests from the start** — the encryption and database services are highly testable but were built without test infrastructure. I'd also consider using `Microsoft.Extensions.DependencyInjection` early, even for a small app, because it makes testing easier. The static `App.SessionKey` pattern works but couples `ViewModels` to the `App` class.

I might also explore using a proper migration system instead of `try/catch ALTER TABLE`, and consider an abstraction layer over the database service (repository pattern) to make it easier to swap implementations in the future.

---

### Q30: How do you stay current with security best practices?

I follow **OWASP guidelines** (the 600k PBKDF2 iteration count comes directly from OWASP 2023 recommendations), read .NET security advisories, and reference the .NET cryptography documentation for correct API usage. Key examples:

- Using `CryptographicOperations.ZeroMemory` instead of `Array.Clear` (the former is JIT-proof)
- Using `FixedTimeEquals` instead of `==` for hash comparisons (prevents timing attacks)
- Using `RandomNumberGenerator` instead of `System.Random` for cryptographic randomness

I also leverage .NET's built-in crypto implementations rather than writing my own — `AesGcm`, `Rfc2898DeriveBytes`, and `RandomNumberGenerator` are all well-audited framework primitives.

---

### Q31: Walk me through how you'd add a new feature — say, TOTP (authenticator codes).

The existing architecture makes this straightforward:

- **Model:** Add `TotpPayload` : `IEntryPayload` with `SecretKey`, `Issuer`, `Algorithm` fields.
- **Templates:** Create XAML `DataTemplates` for view and edit modes, add to `PayloadTemplates.xaml` and the template selectors.
- **Service:** Add `TotpService` in `Services/` implementing RFC 6238 TOTP generation.
- **ViewModel:** Add a computed `CurrentTotpCode` property to `EntryViewModel` that refreshes every 30 seconds using a `DispatcherTimer`. `SecretKey` encrypted with AES-GCM like other sensitive fields.
- **Navigation:** Add "totp" section to sidebar, update badge counts and category filter.

The entire change follows the existing pattern — **no architecture modifications needed**.

---

### Q32: What are the limitations of your current design?

- (1) No cloud sync — intentionally local-only, but some users want cross-device access.
- (2) No browser extension for auto-fill.
- (3) The static `App.SessionKey` pattern means only one session at a time.
- (4) No formal migration system — the `try/catch ALTER TABLE` approach doesn't scale.
- (5) No unit tests yet.
- (6) Windows-only (WPF does not support Linux/macOS).

- **(7)** The managed String limitation means RegisterViewModel's strength-score feature briefly has the password as an immutable GC heap object that cannot be deterministically zeroed.

---

End of Document - Illyrian Vault Interview Q&A Guide